

PHYS 449 - Homework 6, Question 3

Student Name: Allen Mathew

Student ID: 20964892

Due Date: Thursday, November 6, 3:00pm

ChatGPT was used to help generate the code

3. Train an RBM on the MNIST dataset using Contrastive Divergence (CD- k).

(a) Experiment with hyperparameters (number of hidden units, learning rate, value of k , batch size, etc.) to get good performance. Plot the log likelihood as a function of training epoch to justify your choices.

```
In [1]: # === Q3(a): Training RBM on MNIST (hyperparameter tuning and PLL plot) ===
import torch, math, numpy as np, matplotlib.pyplot as plt
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
torch.manual_seed(0); np.random.seed(0)

# --- Load and binarize MNIST ---
transform = transforms.ToTensor()
train_data = datasets.MNIST("./data", train=True, download=True, transform=transform)
test_data = datasets.MNIST("./data", train=False, download=True, transform=transform)
train_loader = DataLoader(train_data, batch_size=256, shuffle=True, drop_last=True)
test_loader = DataLoader(test_data, batch_size=256, shuffle=False)
D = 28*28

def binarize(x): return torch.bernoulli(x)

# --- RBM class ---
class RBM(nn.Module):
    def __init__(self, n_visible, n_hidden):
        super().__init__()
        self.W = nn.Parameter(0.01*torch.randn(n_visible, n_hidden))
        self.a = nn.Parameter(torch.zeros(n_visible))
        self.b = nn.Parameter(torch.zeros(n_hidden))
    def sample_h(self, v):
        p = torch.sigmoid(v @ self.W + self.b)
        return torch.bernoulli(p), p
    def sample_v(self, h):
        p = torch.sigmoid(h @ self.W.t() + self.a)
        return torch.bernoulli(p), p
    def free_energy(self, v):
        term = torch.log1p(torch.exp(v @ self.W + self.b)).sum(1)
```

```

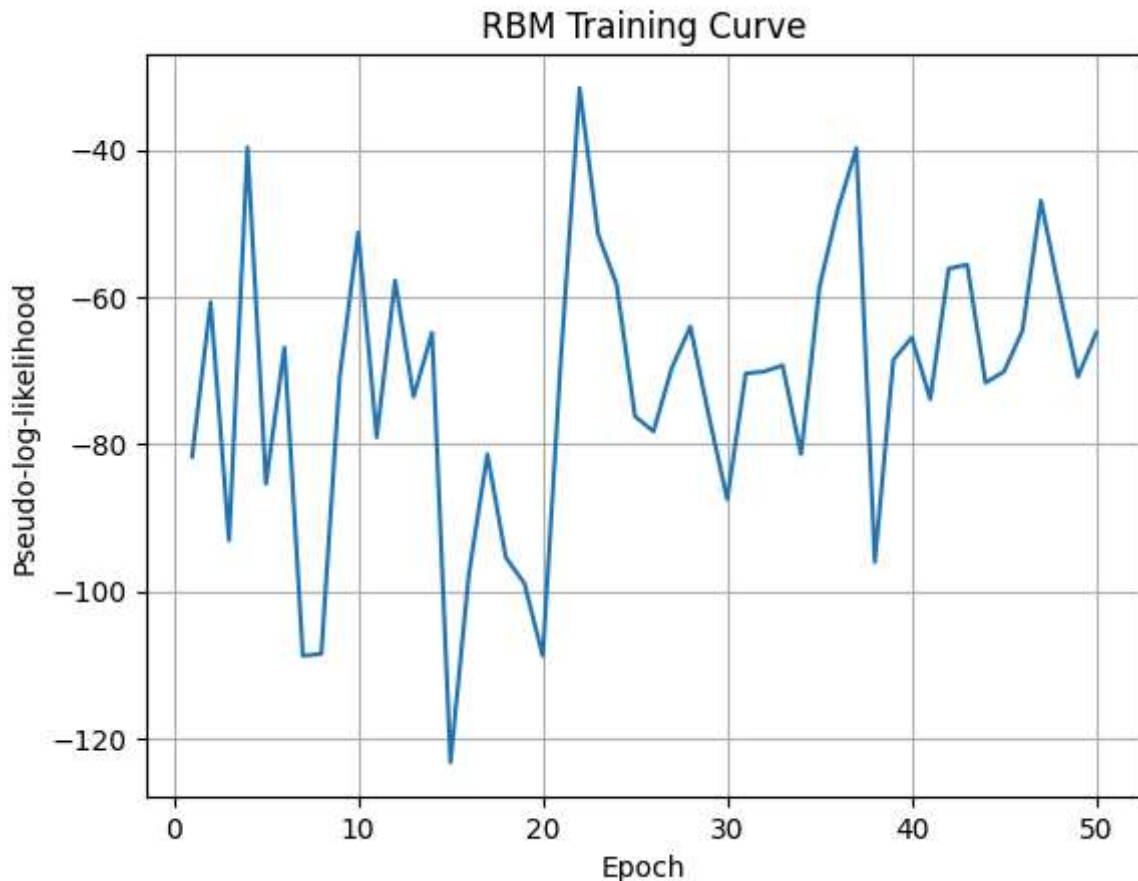
        return -v@self.a - term
def pseudo_log_likelihood(self, v):
    bsz = v.size(0)
    i = torch.randint(0, v.size(1), (bsz,), device=v.device)
    v2 = v.clone(); v2[torch.arange(bsz), i] = 1 - v2[torch.arange(bsz), i]
    return (v.size(1)*torch.log(torch.sigmoid(self.free_energy(v2))-self.free_en

def train_rbm(epochs=10, n_hidden=256, lr=1e-3, k=1):
    rbm = RBM(D, n_hidden).to(device)
    opt = torch.optim.Adam(rbm.parameters(), lr=lr)
    pll_hist = []
    for ep in range(1, epochs+1):
        for (x, _) in train_loader:
            v0 = binarize(x.view(-1, D).to(device))
            h0, _ = rbm.sample_h(v0)
            vk = v0
            for _ in range(k):
                hk, _ = rbm.sample_h(vk)
                vk, _ = rbm.sample_v(hk)
            ph0 = torch.sigmoid(v0 @ rbm.W + rbm.b)
            phk = torch.sigmoid(vk @ rbm.W + rbm.b)
            dw = (v0.t()@ph0 - vk.t()@phk)/v0.size(0)
            da = (v0 - vk).mean(0); db = (ph0 - phk).mean(0)
            rbm.W.grad, rbm.a.grad, rbm.b.grad = -dw, -da, -db
            opt.step(); opt.zero_grad(set_to_none=True)
        with torch.no_grad():
            x_val, _ = next(iter(test_loader))
            pll = rbm.pseudo_log_likelihood(binarize(x_val.view(-1, D).to(device)))
            pll_hist.append(pll); print(f"Epoch {ep:02d} PLL={pll:.3f}")
    plt.plot(range(1, epochs+1), pll_hist)
    plt.xlabel("Epoch"); plt.ylabel("Pseudo-log-likelihood")
    plt.title("RBM Training Curve"); plt.grid(True); plt.show()
    return rbm

rbm = train_rbm(epochs=50, n_hidden=500, lr=0.01, k=5)

```

Epoch 01 PLL=-81.690
Epoch 02 PLL=-60.711
Epoch 03 PLL=-93.067
Epoch 04 PLL=-39.676
Epoch 05 PLL=-85.375
Epoch 06 PLL=-66.854
Epoch 07 PLL=-108.763
Epoch 08 PLL=-108.496
Epoch 09 PLL=-70.804
Epoch 10 PLL=-51.227
Epoch 11 PLL=-79.059
Epoch 12 PLL=-57.768
Epoch 13 PLL=-73.554
Epoch 14 PLL=-64.874
Epoch 15 PLL=-123.238
Epoch 16 PLL=-97.599
Epoch 17 PLL=-81.364
Epoch 18 PLL=-95.409
Epoch 19 PLL=-98.882
Epoch 20 PLL=-108.753
Epoch 21 PLL=-67.683
Epoch 22 PLL=-31.634
Epoch 23 PLL=-51.495
Epoch 24 PLL=-58.277
Epoch 25 PLL=-76.214
Epoch 26 PLL=-78.275
Epoch 27 PLL=-69.619
Epoch 28 PLL=-64.035
Epoch 29 PLL=-76.172
Epoch 30 PLL=-87.497
Epoch 31 PLL=-70.412
Epoch 32 PLL=-70.130
Epoch 33 PLL=-69.305
Epoch 34 PLL=-81.370
Epoch 35 PLL=-58.603
Epoch 36 PLL=-48.039
Epoch 37 PLL=-39.787
Epoch 38 PLL=-96.003
Epoch 39 PLL=-68.565
Epoch 40 PLL=-65.507
Epoch 41 PLL=-73.877
Epoch 42 PLL=-56.155
Epoch 43 PLL=-55.616
Epoch 44 PLL=-71.651
Epoch 45 PLL=-70.138
Epoch 46 PLL=-64.588
Epoch 47 PLL=-46.926
Epoch 48 PLL=-59.292
Epoch 49 PLL=-70.868
Epoch 50 PLL=-64.839



After choosing a variety different combinations, I decided that 500 hidden layers, batch size of 256, 50 epochs, k value of 5, and a learning rate of 0.01 seems th best performance wise and runtime wise. We can see that the PLL curve wildly fluctuates. This is due to CD-k training.

(b) Generate samples of digits from 0 to 9 using Block Gibbs sampling from the trained model. Did your model generalize well?

```
In [2]: # === Q3(b): Sampling digits 0-9 with Block Gibbs ===
from torchvision import utils as tvutils

def show_grid(imgs, nrow=10, title=None):
    grid = tvutils.make_grid(imgs, nrow=nrow, padding=2)
    plt.figure(figsize=(8,8)); plt.axis("off")
    if title: plt.title(title)
    plt.imshow(grid.permute(1,2,0).cpu().numpy()); plt.show()

@torch.no_grad()
def sample_digits(rbm, n_samples=100, steps=500):
    v = torch.bernoulli(0.5*torch.ones(n_samples, rbm.W.size(0), device=device))
    for _ in range(steps):
        h, _ = rbm.sample_h(v); v, _ = rbm.sample_v(h)
    return v.view(-1,1,28,28).cpu()

samples = sample_digits(rbm, n_samples=100, steps=500)
show_grid(samples[:100], nrow=10, title="Generated digits (unconditional)")
```

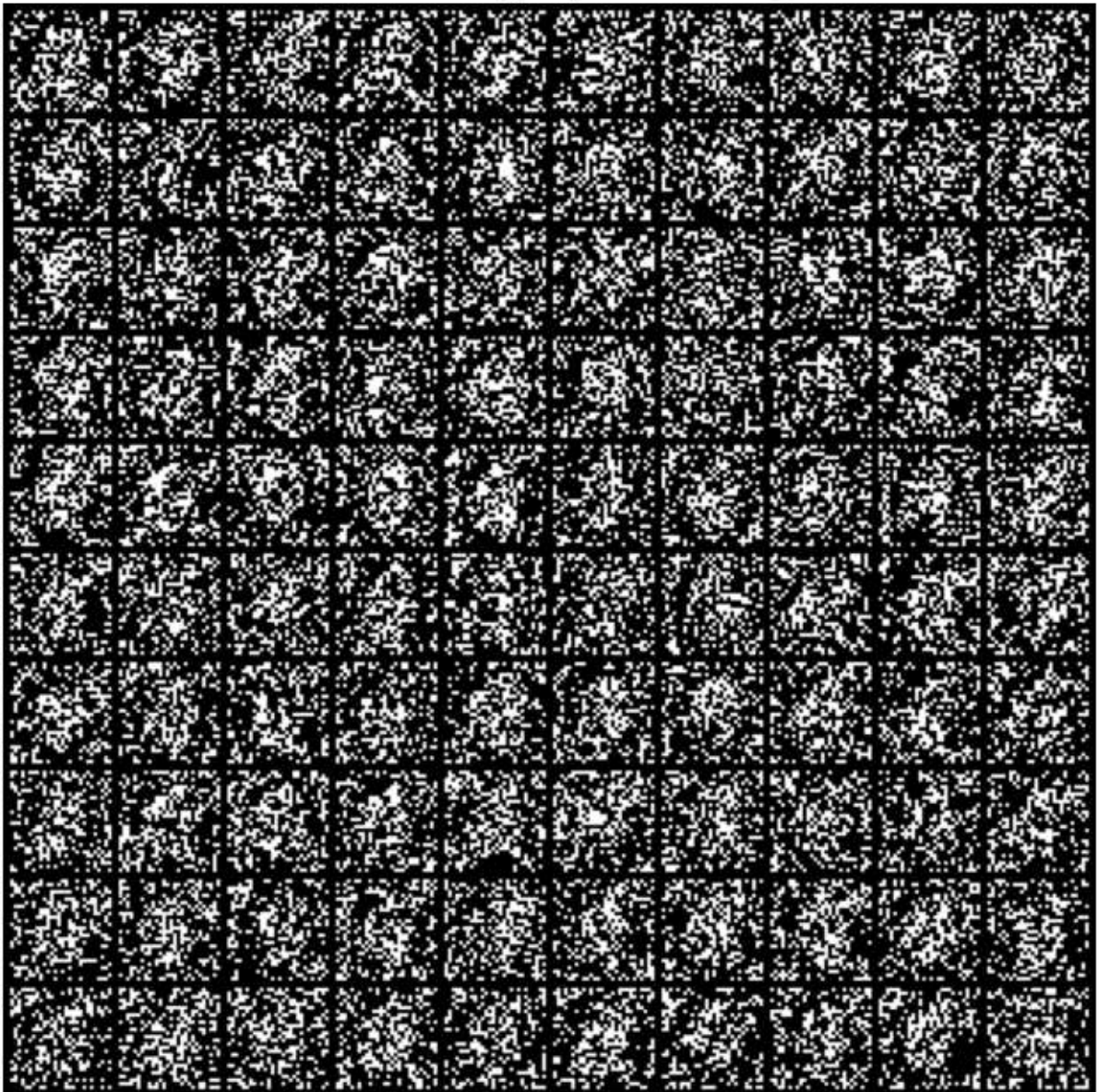
```

# --- seed each chain with a real example 0-9 ---
@torch.no_grad()
def pick_one_per_digit(dataset):
    out=[]
    for d in range(10):
        for x,y in dataset:
            if y==d: out.append(x); break
    return torch.stack(out)

seeds = pick_one_per_digit(datasets.MNIST("./data",train=False,transform=transforms
v = binarize(seeds.view(10,-1).to(device))
for _ in range(200):
    h,_ = rbm.sample_h(v); v,_ = rbm.sample_v(h)
grid = torch.cat([seeds,v.view(-1,1,28,28).cpu()],0)
show_grid(grid,nrow=10,title="Top: real 0-9 Bottom: Gibbs-sampled 0-9")

```

Generated digits (unconditional)



Top: real 0-9 Bottom: Gibbs-sampled 0-9



Based on the results, it seems that my model generalizes somewhat well. The model picks up on patterns in the numbers. But for more complex numbers, it generates more disorderly numbers. Specifically, numbers 0, 1, and 2 are generated well. But the other numbers are very hard to distinguish since they are more complex and can be easily written poorly in the MNIST dataset.

(c) For each hidden unit, reshape its weights into a 28×28 matrix and plot. Discuss any features descendent from the original images captured by the hidden units.

In [3]: `# === Q3(c): Visualize hidden-unit filters (fixed version) ===`

```
W = rbm.W.detach().cpu().T
imgs = torch.tensor(W).view(-1, 1, 28, 28)

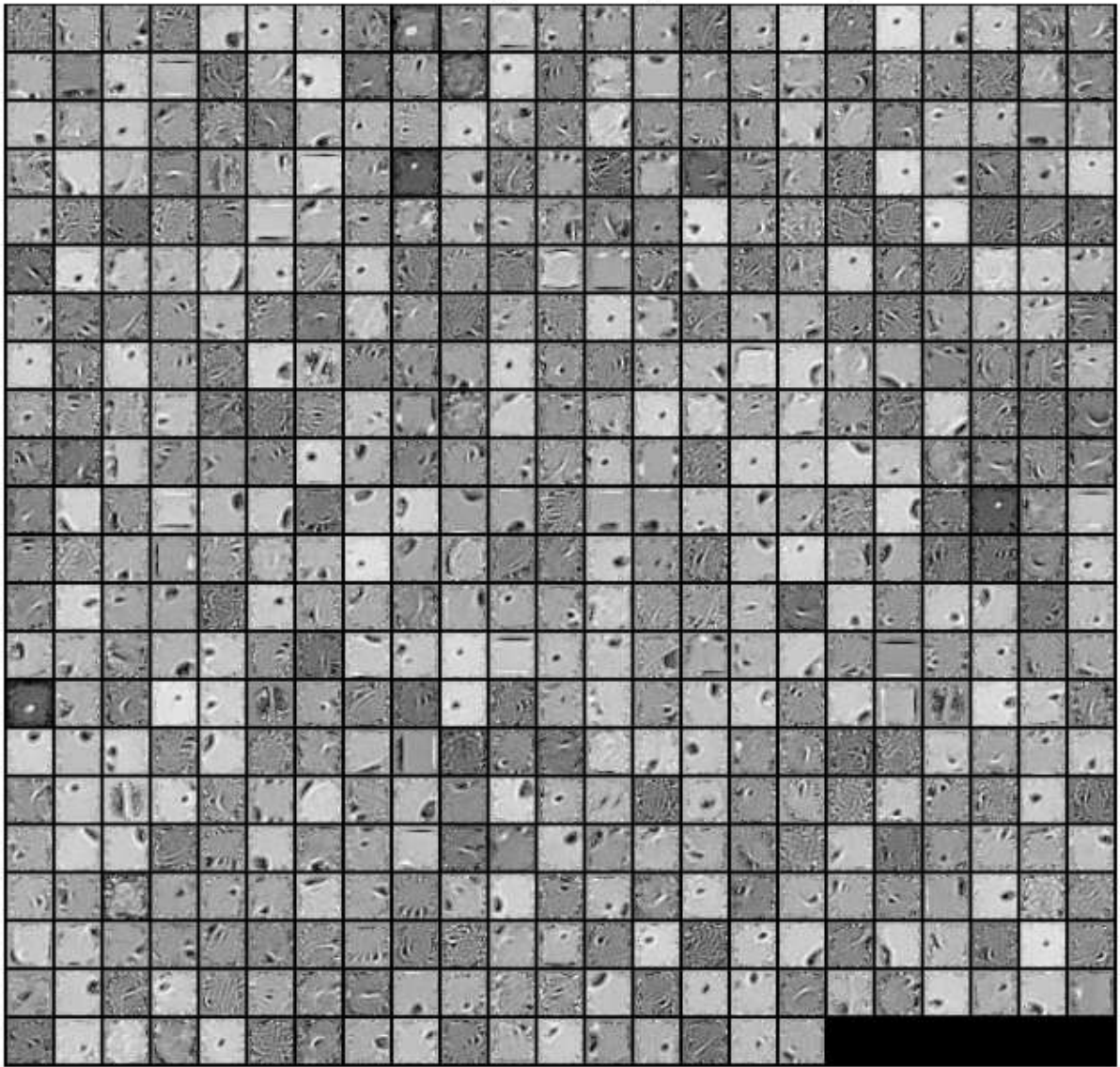
# normalize each filter using torch.amin / torch.amax
mins = torch.amin(imgs, dim=(2, 3), keepdim=True)
maxs = torch.amax(imgs, dim=(2, 3), keepdim=True)
imgs = (imgs - mins) / (maxs - mins + 1e-8)

side = int(math.ceil(math.sqrt(imgs.size(0))))
show_grid(imgs, nrow=side, title="Hidden unit weight filters (28x28 each)")
```

C:\Users\Allen\AppData\Local\Temp\ipykernel_10488\1735787205.py:3: UserWarning: To copy construct from a tensor, it is recommended to use `sourceTensor.detach().clone()` or `sourceTensor.detach().clone().requires_grad_(True)`, rather than `torch.tensor(sourceTensor)`.

```
imgs = torch.tensor(W).view(-1, 1, 28, 28)
```

Hidden unit weight filters (28x28 each)



We can see that a lot of the filters have certain stroke patterns that can be found in MNIST numbers. This includes curves, horizontal and vertical lines, as well as diagonals. Specifically, the vertical lines correspond to 1, the circular curves correspond to 0 or 6, other more complex curves correspond to the other numbers. It's also important to note that the filters only give a hint of what the number is since it only really captures dominant characteristics/stroke patterns and not the whole number.